Setup Two GitHub Identities (Work + Personal)

Goal

Use two separate GitHub accounts on the same machine:

```
Work GitHub
├── Work SSH Key
├── Work Email
└── Work Repositories

Personal GitHub
├── Personal SSH Key
├── Personal Email
└── Personal Repositories
```

This setup works on:

- macOS
- Linux
- Windows (Git Bash)

Architecture

Git uses three separate concepts:

```
SSH Key
    ↓
Authentication to GitHub

Git Email
    ↓
Commit Identity

Remote URL
    ↓
Determines which SSH key is used
```

Example:

```
github-work
    ↓
Work SSH Key
    ↓
GAmanvardhanhararіya

github-personal
    ↓
Personal SSH Key
    ↓
yo-msh
```

---

1. Generate SSH Keys

Work Key

```
ssh-keygen -t ed25519 -C "work@company.com"
```

Save as:

```
~/.ssh/id_ed25519_work
```

Personal Key

```
ssh-keygen -t ed25519 -C "personal@gmail.com"
```

Save as:

```
~/.ssh/id_ed25519_personal
```

---

2. Add Keys to SSH Agent

macOS

```
eval "$(ssh-agent -s)"

ssh-add --apple-use-keychain ~/.ssh/id_ed25519_work
ssh-add --apple-use-keychain ~/.ssh/id_ed25519_personal
```

If unavailable:

```
ssh-add ~/.ssh/id_ed25519_work
ssh-add ~/.ssh/id_ed25519_personal
```

Linux

```
eval "$(ssh-agent -s)"

ssh-add ~/.ssh/id_ed25519_work
ssh-add ~/.ssh/id_ed25519_personal
```

Windows (Git Bash)

```
eval $(ssh-agent -s)

ssh-add ~/.ssh/id_ed25519_work
ssh-add ~/.ssh/id_ed25519_personal
```

---

3. Add Public Keys to GitHub

Display public key:

```
cat ~/.ssh/id_ed25519_work.pub
```

or

```
cat ~/.ssh/id_ed25519_personal.pub
```

macOS clipboard:

```
pbcopy < ~/.ssh/id_ed25519_personal.pub
```

Windows clipboard:

```
cat ~/.ssh/id_ed25519_personal.pub | clip
```

Add to:

```
GitHub
→ Settings
→ SSH and GPG Keys
→ New SSH Key
```

---

4. Configure SSH Aliases

Create:

```
nano ~/.ssh/config
```

```
Host github-work
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_work
  AddKeysToAgent yes
  IdentitiesOnly yes

Host github-personal
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_personal
```

```
  AddKeysToAgent yes
  IdentitiesOnly yes
```

macOS only:

```
UseKeychain yes
```

can also be added.

---

5. Verify Authentication

```
ssh -T git@github-work
```

Expected:

```
Hi <work-username>!
```

```
ssh -T git@github-personal
```

Expected:

```
Hi <personal-username>!
```

Verbose debugging:

```
ssh -vT git@github-work
```

---

6. Organize Repositories

Recommended structure:

```
~/work-projects/
├── repo1
├── repo2
```

```
~/personal-projects/
├── portfolio
├── blog
├── side-project
```

Example:

```
/Users/maan/work-projects
/Users/maan/personal-projects
```

---

7. Directory-Based Git Identity

Main config:

```
nano ~/.gitconfig
```

```
[includeIf "gitdir:/Users/maan/work-projects/"]
    path = ~/.gitconfig-work

[includeIf "gitdir:/Users/maan/personal-projects/"]
    path = ~/.gitconfig-personal
```

Work Identity

Create:

```
nano ~/.gitconfig-work
```

```
[user]
    name = Your Name
    email = work@company.com
```

Personal Identity

Create:

```
nano ~/.gitconfig-personal
```

```
[user]
    name = Your Name
    email = personal@gmail.com
```

---

8. Clone Repositories Correctly

Work

```
git clone git@github-work:ORG_NAME/repository.git
```

Personal

```
git clone git@github-personal:USERNAME/repository.git
```

Never use:

```
git@github.com:user/repo.git
```

for multi-account setups.

---

9. Verify Git Identity

Inside repository:

```
git config user.name
git config user.email
```

Show source:

```
git config --show-origin --get user.email
```

Expected:

```
file:/Users/user/.gitconfig-work
```

or

```
file:/Users/user/.gitconfig-personal
```

Troubleshooting

Wrong GitHub Account

Check:

```
ssh -T git@github-work
ssh -T git@github-personal
```

Check Active Keys

```
ssh-add -l
```

Check Which Key SSH Uses

```
ssh -vT git@github-work
```

Look for:

```
Offering public key:
```

Check Remote URL

```
git remote -v
```

Fix:

```
git remote set-url origin git@github-work:ORG/repo.git
```

or

```
git remote set-url origin git@github-personal:user/repo.git
```

Check Current Repository

```
git rev-parse --show-toplevel
```

Check Repository Identity

```
git config --show-origin --get user.email
```

Useful Git Aliases

```
[alias]
    st = status
    br = branch
    co = checkout
    sw = switch
    ci = commit

    last = log -1 HEAD

    unstage = restore --staged
    amend = commit --amend
```

```
    lg = log --oneline --graph --decorate --all

    ll = log --graph --decorate --pretty=format:'%C(yellow)%h%Creset - %C(cyan)
%an%Creset %s %C(green)(%cr)%Creset' --all

    hist = log --pretty=format:'%h %ad | %s%d [%an]' --graph --date=short

    recent = for-each-ref --sort=-committerdate refs/heads/ --format='%
(refname:short)'

    aliases = config --get-regexp '^alias\\.'
```

Useful commands:

```
git st
git lg
git ll
git hist
git aliases
```

Final Verification Checklist

```
✓ Work SSH key created
✓ Personal SSH key created

✓ github-work alias configured
✓ github-personal alias configured

✓ Work key added to Work GitHub
✓ Personal key added to Personal GitHub

✓ Directory-based Git config enabled

✓ Work repos cloned using github-work
✓ Personal repos cloned using github-personal

✓ git config --show-origin works

✓ ssh -T works for both accounts
```

At this point you can push and pull from both GitHub accounts indefinitely without manually changing Git configuration per repository.